

Universidade do Minho

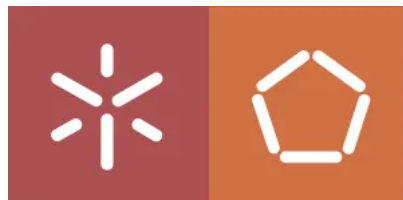
Programação Orientada a Objetos

Trabalho Prático - DomusControl

2025/2026

Grupo 48

Diogo Silva - a104183
Miguel Barrocas - a104272



Universidade do Minho
Escola de Engenharia

Conteúdo

1	Introdução	2
1.1	Objetivos do Sistema	2
2	Arquitetura e Estrutura de Classes	3
2.1	Classe Dispositivo	3
2.2	Classe Lampada	4
2.3	Classe Sensor	4
2.4	Outros dispositivos	5
3	Gestão de Casas e Divisões	6
3.1	Classe Casa	6
3.2	Classe Divisao	7
4	Sistema de Utilizadores	7
4.1	Classe Utilizador	7
4.2	Classe Administrador	8
4.3	Classe UtilizadorComum	8
5	Sistema de Automações	9
5.1	Classe Automacao	9
6	Sistema de Escalonamentos	10
6.1	Classe Escalonamento	10
6.1.1	Exemplo de Escalonamento	10
7	Sistema de Cenários	11
7.1	Classe Cenario	11
7.1.1	Exemplos de Cenários	11
8	Controlador Principal	12
8.1	Classe DomusControl	12
9	Diagrama de Classes	13
10	Descrição de Funcionalidades	14
11	Conclusão	17

1 Introdução

Este relatório descreve o desenvolvimento do sistema DomusControl, uma aplicação de gestão doméstica implementada em Java que permite o controlo de dispositivos inteligentes em ambientes residenciais. O projeto foi desenvolvido no âmbito da unidade curricular de Programação Orientada a Objetos, com o objetivo de aplicar os conceitos fundamentais deste paradigma de programação na resolução de um problema real e complexo.

A crescente adoção de tecnologias de automação residencial tem criado a necessidade de sistemas integrados capazes de gerir múltiplos dispositivos de forma eficiente e intuitiva. O DomusControl surge como resposta a esta necessidade, oferecendo uma plataforma completa que não só permite o controlo manual de dispositivos, mas também implementa sistemas avançados de automação baseados em sensores, escalonamentos temporais e cenários.

O sistema foi concebido tendo em consideração os princípios da Programação Orientada a Objetos. Esta abordagem permitiu criar uma arquitetura modular e extensível, facilitando a manutenção do código e a adição de novas funcionalidades.

1.1 Objetivos do Sistema

O principal objetivo do DomusControl é proporcionar uma plataforma integrada para:

- Gestão de múltiplas casas e dispositivos inteligentes;
- Automação baseada em sensores e escalonamentos temporais;
- Criação e ativação de cenários personalizados;
- Análise estatística do consumo energético;
- Controlo de acesso através de sistema de utilizadores.

2 Arquitetura e Estrutura de Classes

O sistema DomusControl foi desenvolvido seguindo os princípios da Programação Orientada a Objetos, com uma arquitetura em camadas bem definida. O sistema tem a seguinte arquitetura:

- **model:** Contém as classes de domínio (Casa, Divisão, Cenário, Escalonamento, Dispositivos, Utilizadores e Automações);
- **controller:** Onde se encontra a lógica (DomusControl);
- **view:** Contém a interface de utilizador (TextUI);
- **exceptions:** Exceções personalizadas do sistema.

2.1 Classe Dispositivo

A classe `Dispositivo` é uma classe abstrata que representa o comportamento genérico de todos os dispositivos inteligentes do sistema.

```
1 public abstract class Dispositivo implements Serializable {
2     private static int contadorId = 1;
3     private int id;
4     private String marca;
5     private String modelo;
6     private double consumoWh;
7     private boolean ligado;
8     private int numAtivacoes;
9     private double horasLigado;
10
11     public abstract void ligar();
12     public abstract void desligar();
13     public abstract String getEstadoDetalhado();
14 }
```

Listing 1: Estrutura da classe `Dispositivo`

A classe `Dispositivo` é a base de todos os dispositivos inteligentes do sistema DomusControl. Trata-se de uma classe abstrata que encapsula o comportamento e as características comuns de todos os tipos de dispositivos.

Esta abordagem permite ao sistema tratar diferentes tipos de dispositivos de forma uniforme sem alterar as características dos mesmos. Por exemplo, tanto uma lâmpada como uma televisão podem ser ligadas e desligadas, mas cada uma implementa estes comportamentos de forma distinta e possui atributos específicos.

As principais responsabilidades desta classe incluem a gestão de identificação única de dispositivos através de um contador, a verificação do estado atual (ligado/desligado), a contabilização de horas de funcionamento, o número de ativações e o armazenamento de informação técnica como marca, modelo e consumo energético. Estes dados são essenciais para as funcionalidades de estatísticas e análise de consumo do sistema.

São definidos métodos abstratos como `ligar()`, `desligar()` e `getEstadoDetalhado()` que todas as subclasses devem implementar. Esta abordagem garante que qualquer novo

tipo de dispositivo adicionado ao sistema terá sempre as operações básicas disponíveis, mantendo a consistência da interface.

2.2 Classe Lampada

```
1 public class Lampada extends Dispositivo {
2     public static final int COR_MIN = 2700;
3     public static final int COR_MAX = 4000;
4
5     private int intensidade;
6     private int corKelvin;
7
8     @Override
9     public void ligar() {
10         incrementarAtivacoes();
11         setLigado(true);
12         if(this.intensidade == 0) {
13             this.intensidade = 100;
14         }
15     }
16 }
```

Listing 2: Estrutura da classe Lampada

A classe Lampada é uma subclasse de Dispositivo que tem todas as características de uma lâmpada. Além dos atributos base, adiciona características específicas como intensidade luminosa (0-100%) e temperatura de cor (2700-4000K). No método ligar(): se a intensidade estiver a zero, ajusta automaticamente para 100% evitando uma lâmpada ligada mas sem emitir luz. Temos ainda o método getEstadoDetalhado() que retorna informação sobre a intensidade e a cor da lâmpada.

2.3 Classe Sensor

A classe ‘Sensor’ representa um tipo especial de dispositivo no sistema DomusControl. Enquanto dispositivos como lâmpadas e cortinas alteram o ambiente, os sensores capturam dados que servem de base para o sistema de automações, permitindo que o DomusControl tome decisões inteligentes baseadas em condições reais.

Cada sensor é caracterizado por três atributos fundamentais: o tipo de grandeza medida (temperatura, luminosidade, pluviosidade, etc.), a unidade de medida correspondente (°C, lux, mm/h) e o valor atual lido. Esta estrutura genérica permite ao sistema suportar qualquer tipo de sensor sem necessidade de criar subclasses específicas, bastando configurar adequadamente estes três parâmetros.

```
1 @Override
2 public void desligar() {
3     setLigado(false);
4     this.valorAtual = 0.0;
5 }
```

Listing 3: Comportamento específico do método desligar()

Um aspecto importante da arquitetura é que os sensores só fornecem leituras quando estão ligados. Ao desligar um sensor, o método `desligar()` não apenas altera o estado mas também reinicia o valor atual para zero. Este comportamento é essencial para a integridade do sistema de automações, que verifica explicitamente se um sensor está ligado antes de utilizar os seus valores.

2.4 Outros dispositivos

O sistema inclui ainda as classes `Cortina` e `PortaoGaragem`, ambas com funcionalidade de abertura regulável (0-100%), e a classe `Rele`, um dispositivo de on/off sem configurações adicionais.

Estas três classes seguem a mesma arquitetura e polimorfismo que as descritas anteriormente, especializando o comportamento base de `Dispositivo` conforme as suas necessidades específicas.

Adicionalmente, implementamos a classe `Televisao` como um dispositivo extra não especificado no enunciado original, com controlo de canal e volume. Optamos por não implementar mais tipos de dispositivos, pois não consideramos que o objetivo principal do trabalho era a criação de tipos de dispositivos semelhantes.

3 Gestão de Casas e Divisões

3.1 Classe Casa

```
1 public class Casa implements Serializable {
2     private static int contadorId = 1;
3     private String id;
4     private String nome;
5     private String morada;
6     private Map<String, Divisao> divisoes;
7
8     public double consumoTotal() {
9         double consumoT = 0.0;
10        for(Divisao dv : this.divisoes.values()) {
11            for(Dispositivo disp : dv.getDispositivos()) {
12                consumoT += disp.getConsumoWh() * disp.getHorasLigado();
13            }
14        }
15        return consumoT;
16    }
17 }
```

Listing 4: Estrutura da classe Casa

A classe Casa representa uma habitação no sistema DomusControl, agregando divisões e fornecendo métodos para gestão de dispositivos. Utiliza um `Map<String, Divisao>` para organizar as divisões por nome, permitindo acesso eficiente e evitando duplicações. Cada casa é identificada unicamente através de um ID gerado automaticamente por um contador com formato C1, C2, etc.

A arquitetura implementa o padrão de encapsulamento: todos os métodos que retornam divisões ou recebem divisões como parâmetro trabalham com cópias, protegendo o estado interno da casa contra modificações externas não controladas. Este padrão repete-se em toda a hierarquia, garantindo integridade dos dados.

A classe fornece funcionalidades estatísticas importantes como `consumoTotal()`, que agrega o consumo de todos os dispositivos da casa. A função `acumularHoras()` propaga o avanço temporal para todas as divisões e respetivos dispositivos. Estas funcionalidades são essenciais para o sistema de simulação temporal e análise energética, permitindo identificar padrões de consumo e otimizar a utilização dos dispositivos.

3.2 Classe Divisao

```
1 public class Divisao implements Serializable {
2     private String nome;
3     private List<Dispositivo> dispositivos;
4
5     public void acumularHoras(double horas) {
6         for(Dispositivo d : this.dispositivos) {
7             d.addHorasLigado(horas);
8         }
9     }
10 }
```

Listing 5: Estrutura da classe Divisao

A classe Divisao representa uma divisão (sala, quarto, cozinha...) dentro de uma casa, funcionando como contentor de dispositivos. Utiliza uma `List<Dispositivo>` para armazenar os dispositivos, permitindo múltiplos dispositivos do mesmo tipo numa divisão e mantendo a ordem de inserção.

A arquitetura implementa dois métodos de acesso a dispositivos: `getDispositivo()` que retorna uma cópia para uso externo e `getDispositivoInterno()` que retorna a referência direta, usado apenas internamente pela classe Casa para operações de ligar/desligar. Esta distinção é crucial para manter o encapsulamento enquanto otimiza operações frequentes.

O método `acumularHoras()` é fundamental para o sistema de simulação temporal, propagando o avanço de tempo para todos os dispositivos da divisão. Quando o utilizador avança 2 horas no sistema, este método garante que todos os dispositivos ligados acumulam essas 2 horas no seu contador, permitindo cálculos precisos de consumo energético.

4 Sistema de Utilizadores

4.1 Classe Utilizador

```
1 public abstract class Utilizador implements Serializable {
2     private static int contadorId = 1;
3     private String id;
4     private String nome;
5     private String email;
6     private String password;
7     private Set<String> casas;
8
9     public abstract boolean isAdministrador();
10 }
```

Listing 6: Estrutura da classe Utilizador

A classe Utilizador é uma classe abstrata que representa os utilizadores do sistema servindo de base para dois tipos concretos: Administrador e UtilizadorComum. Cada utilizador recebe um ID único gerado automaticamente no formato U1, U2, U3..., através de um contador partilhado por todas as instâncias.

A arquitetura utiliza um `Set<String>` para armazenar os IDs das casas às quais o utilizador tem acesso. A escolha de Set em vez de List é porque garante que não existem

duplicações (um utilizador não pode ter acesso à mesma casa duas vezes) e permite verificação eficiente através do método `temAcessoCasa()`. Esta estrutura suporta o modelo de permissões do sistema, onde utilizadores podem gerir múltiplas habitações.

O método `isAdministrador()` força as subclasses a definirem o seu tipo mas permite que a classe base utilize esta informação (por exemplo, no método `toString()`). A password é armazenada em texto simples, sendo uma área identificada para melhoria futura.

4.2 Classe Administrador

```
1 public class Administrador extends Utilizador {  
2     @Override  
3     public boolean isAdministrador() {  
4         return true;  
5     }  
6 }
```

Listing 7: Estrutura da classe Administrador

4.3 Classe UtilizadorComum

```
1 public class UtilizadorComum extends Utilizador {  
2     @Override  
3     public boolean isAdministrador() {  
4         return false;  
5     }  
6 }
```

Listing 8: Estrutura da classe UtilizadorComum

As classes `Administrador` e `UtilizadorComum` são subclasses concretas de `Utilizador`, diferenciando-se apenas na implementação do método `isAdministrador()` que retorna `true` e `false` respetivamente.

5 Sistema de Automações

5.1 Classe Automacao

```
1 public abstract class Automacao implements Serializable {  
2     private String nome;  
3     private boolean ativa;  
4     private List<AcaoDispositivo> acoes;  
5  
6     public void addAcao(AcaoDispositivo acao) { ... }  
7     public void removeAcao(int index) { ... }  
8     public abstract boolean avaliar(DomusControl sistema);  
9     public void executar(DomusControl sistema) { ... }  
10 }
```

Listing 9: Estrutura da classe Automacao

A classe Automacao é uma classe abstrata que implementa o sistema de automações do DomusControl, permitindo executar ações sobre dispositivos automaticamente quando determinadas condições são satisfeitas. Cada automação possui um nome identificativo, um estado (ativa/inativa) e uma lista de ações a executar.

A arquitetura separa claramente duas responsabilidades através de métodos distintos: avaliar() verifica se as condições para ativar a automação estão reunidas, enquanto executar() executa todas as ações associadas caso a automação esteja ativa. Esta separação permite diferentes tipos de automações partilharem a mesma lógica de execução, variando apenas nas suas condições de ativação.

6 Sistema de Escalonamentos

6.1 Classe Escalonamento

Permite agendar ações para horários específicos.

```
1 public class Escalonamento implements Serializable {
2     private String nome;
3     private LocalDateTime horaInicio;
4     private LocalDateTime horaFim;
5     private Set<DayOfWeek> diasSemana;
6     private List<AcaoDispositivo> acoes;
7     private boolean ativo;
8
9     public boolean estaAtivo(LocalDateTime dataHora) {
10         if(!this.ativo) return false;
11         DayOfWeek diaAtual = dataHora.getDayOfWeek();
12         LocalDateTime horaAtual = dataHora.toLocalTime();
13         return this.diasSemana.contains(diaAtual) &&
14             !horaAtual.isBefore(this.horaInicio) &&
15             !horaAtual.isAfter(this.horaFim);
16     }
17 }
```

Listing 10: Estrutura da classe Escalonamento

6.1.1 Exemplo de Escalonamento

```
1 Set<DayOfWeek> diasUteis = new HashSet<>(Arrays.asList(
2     DayOfWeek.MONDAY, DayOfWeek.TUESDAY,
3     DayOfWeek.WEDNESDAY,
4     DayOfWeek.THURSDAY, DayOfWeek.FRIDAY
5 ));
6
7 Escalonamento esc1 = new Escalonamento(
8     "Luz sala a noite",
9     LocalDateTime.of(19, 0),
10    LocalDateTime.of(23, 0),
11    diasUteis
12 );
13
14 esc1.addAcao(new AcaoDispositivo(
15     1,
16     "Ligar lampada da sala",
17     d -> d.ligar()
18 ));
19
20 model.addEscalonamento(esc1);
```

Listing 11: Exemplo: Luz da sala em dias úteis

7 Sistema de Cenários

7.1 Classe Cenario

A classe `Cenario` permite fazer várias ações sobre dispositivos que pode ser ativada instantaneamente com um único comando. Diferencia-se das automações por não ter condições de ativação (método `avaliar()`) nem estado ativo/inativo, sendo executada apenas pelo método `ativar()`. Exemplos práticos incluem cenários como "Fora de Casa" (desliga todos os dispositivos e fecha cortinas) ou "Acordar", proporcionando conveniência e rapidez na gestão do ambiente doméstico.

```
1 public class Cenario implements Serializable {
2     private String nome;
3     private String descricao;
4     private List<AcaoDispositivo> acoes;
5
6     public void addAcao(AcaoDispositivo acao){
7         ...
8     }
9     public void removeAcao(int index){
10        ...
11    }
12    public void ativar(DomusControl sistema){
13        ...
14    }
15
16 }
```

Listing 12: Estrutura da classe `Cenario`

7.1.1 Exemplos de Cenários

```
1 Cenario foraDeCasa = new Cenario(
2     "Fora de Casa",
3     "Desliga tudo e fecha as cortinas"
4 );
5
6 foraDeCasa.addAcao(new AcaoDispositivo(
7     1, "Desligar lampada da sala", d -> d.desligar()));
8 foraDeCasa.addAcao(new AcaoDispositivo(
9     2, "Desligar coluna", d -> d.desligar()));
10 foraDeCasa.addAcao(new AcaoDispositivo(
11     3, "Desligar Televisao", d -> d.desligar()));
12 foraDeCasa.addAcao(new AcaoDispositivo(
13     4, "Fechar cortina da sala", d -> d.desligar()));
14 foraDeCasa.addAcao(new AcaoDispositivo(
15     5, "Fechar portao", d -> d.desligar()));
16
17 model.addCenario(foraDeCasa);
```

Listing 13: Cenário: Fora de Casa

8 Controlador Principal

8.1 Classe DomusControl

A classe `DomusControl` é o coração do sistema, gerindo todas as entidades e operações.

```
1 public class DomusControl implements Serializable {  
2     private int contadorDispositivos;  
3     private int contadorUtilizadores;  
4     private int contadorCasas;  
5     private Map<String, Utilizador> utilizadores;  
6     private Map<String, Casa> casas;  
7     private LocalDateTime dataHoraAtual;  
8     private List<Automacao> automacoes;  
9     private LocalDateTime ultimaVerificacao;  
10    private List<Escalonamento> escalonamentos;  
11    private List<Cenario> cenarios;  
12 }
```

Listing 14: Principais atributos do `DomusControl`

A classe `DomusControl` é quem controla o sistema e implementa um padrão arquitetural MVC (Model View Controller) ao coordenar todas as operações entre o modelo de domínio e a interface de utilizador. Armazena e gere cinco coleções principais: utilizadores (`Map<String, Utilizador>`), casas (`Map<String, Casa>`), automações (`List<Automacao>`), escalonamentos (`List<Escalonamento>`) e cenários (`List<Cenario>`).

A arquitetura implementa encapsulamento através de clonagem em todos os métodos de acesso: quando um objeto é adicionado ao sistema, uma cópia é armazenada; quando é recuperado, uma cópia é retornada. Este padrão, aplicado consistentemente em todos os getters e setters, garante que o estado interno do controlador nunca pode ser modificado diretamente por código externo, e assim, protege a integridade dos dados e facilita o debugging.

Um dos aspetos mais sofisticados é o sistema de simulação temporal. O método `avancarTempo()` não apenas avança no relógio do sistema (`dataHoraAtual`), mas propaga este avanço através de toda a hierarquia: calcula as horas avançadas, invoca `acumularHoras()` que percorre todas as casas, divisões e dispositivos atualizando os contadores de tempo ligado e por fim verifica e executa automações e escalonamentos que possam ter sido ativados neste novo momento temporal.

Métodos estatísticos: O método `top3DivisoesComMaisDispositivos()` recolhe todas as divisões de todas as casas, ordena-as por número de dispositivos de forma decrescente e retorna as três primeiras. Já o método `casaQueMaisConsome()` percorre a hierarquia completa (`Casa` → `Divisão` → `Dispositivo`) calculando o consumo total através da fórmula $\text{consumoWh} \times \text{horasLigado}$ para cada dispositivo, retornando a casa com maior consumo acumulado.

9 Diagrama de Classes

Disponível na pasta de trabalho em .png

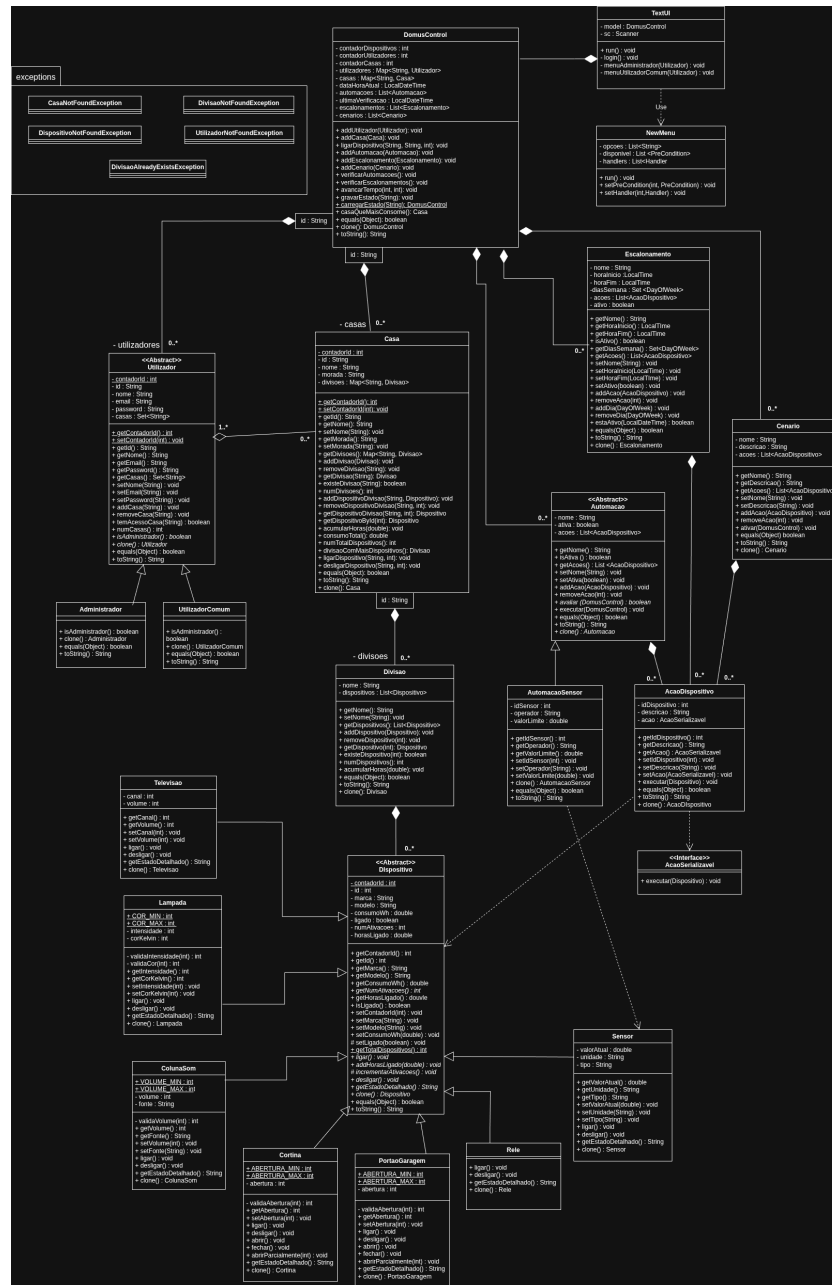


Figura 1: Diagrama de classes

Para não sobrecarregar o diagrama apenas colocamos alguns métodos principais de DomusControl e de TextUI.

10 Descrição de Funcionalidades

No arranque do programa o utilizador encontra o seguinte menu:

```
1 - Carregar estado de ficheiro
2 - Carregar estado de teste
OPÇÃO: 
```

Figura 2: Menu Inicial

O sistema oferece duas opções de inicialização para flexibilizar o uso e facilitar os testes: A opção 1 (Carregar estado de ficheiro) permite recuperar um estado guardado anteriormente. Quando o utilizador escolhe guardar o estado do sistema (através da opção correspondente no menu), toda a informação é persistida no ficheiro estado.dat. Esta opção carrega esse ficheiro, restaurando completamente o estado anterior incluindo utilizadores, casas, dispositivos, automações, escalonamentos e cenários exatamente como estavam no momento da gravação. É particularmente útil para continuar trabalho anterior ou manter configurações entre diferentes sessões de utilização.

A opção 2 (Carregar estado de teste) executa o método `carregarEstadoTeste()` que cria programaticamente um administrador apenas.

Após o utilizador seleccionar uma opção e prosseguir com o seu login, no caso de um Administrador, aparecerá o seguinte menu:

```
OPÇÃO: 1

ID: U1
Password: 1234
Bem-vind@, Miguel Barrocas

**** DomusControl ****

1 - Gestão de Utilizadores
2 - Gestão de Casas
3 - Gestão de Dispositivos
4 - Estatísticas
5 - Cenários
6 - Automações
7 - Escalonamentos
8 - Avançar Tempo
9 - Guardar Estado
0 - Sair

OPÇÃO: 
```

Figura 3: MenuAdmin

Como podemos verificar na Figura3, após autenticação bem-sucedida, o sistema apresenta um menu principal adaptado ao tipo de utilizador. O menu de administrador oferece acesso completo a todas as funcionalidades do sistema, organizadas em nove opções.

A Gestão de Utilizadores permite criar novos utilizadores (administradores ou comuns), remover utilizadores existentes e listar todos os utilizadores registados no sistema.

A Gestão de Casas oferece funcionalidades para adicionar e remover casas, listar todas as casas registradas, visualizar detalhes de uma casa específica, adicionar e remover divisões, e associar utilizadores a casas para controlo de acessos. A Gestão de Dispositivos centraliza todas as operações sobre dispositivos: adicionar novos dispositivos a divisões específicas, remover dispositivos, ligar e desligar dispositivos individualmente, listar todos os dispositivos de uma divisão e regular parâmetros específicos de cada tipo.

O menu de Estatísticas disponibiliza análises do sistema incluindo a identificação da casa com maior consumo energético, top 3 de dispositivos mais utilizados (por tempo de funcionamento e por número de ativações), top 3 de divisões com mais dispositivos, e listagem de todos os dispositivos atualmente ligados numa casa.

As opções de Cenários, Automações e Escalonamentos permitem criar, listar, ativar/-desativar e remover cada um destes elementos do sistema de automação doméstica. A funcionalidade Avançar Tempo é crucial para testes, permitindo simular o passar de horas ou dias instantaneamente e observar o comportamento de automações e escalonamentos sem esperar pelo tempo real. Finalmente, a opção Guardar Estado persiste toda a informação do sistema através de serialização, garantindo que nada se perde entre sessões.

Após escolhermos, por exemplo, a opção 1: Gestão de Utilizadores aparece o seguinte:

```
1 - Adicionar Utilizador
2 - Remover Utilizador
3 - Listar Utilizadores
0 - Sair

OPÇÃO: 3

ID      : U1
Nome    : Miguel Barrocas
Email   : barrocas@sapo.pt
Tipo    : Administrador
Casas   : 1
-----
ID      : U2
Nome    : Diogo Silva
Email   : diogo@gmail.com
Tipo    : Utilizador comum
Casas   : 2
-----
```

Figura 4: Gestão de Utilizadores

A funcionalidade de Gestão de Utilizadores é exclusiva do perfil de administrador e reflete o modelo de controlo de acessos implementado no sistema. Esta secção permite realizar três operações fundamentais sobre os utilizadores do DomusControl.

A opção Adicionar Utilizador solicita ao administrador o nome, email e password do novo utilizador, seguido da escolha do tipo (Administrador ou Utilizador Comum). O sistema gera automaticamente um identificador único no formato "U1", "U2", etc., que é apresentado ao administrador após a criação. Este ID é essencial para futuras operações de autenticação, uma vez que o login é realizado através do identificador e não do email.

A opção Remover Utilizador permite eliminar um utilizador do sistema mediante a introdução do seu ID. Por segurança, o sistema solicita confirmação explícita antes de proceder à remoção, prevenindo eliminações acidentais. Se o utilizador não existir, é lançada a exceção `UtilizadorNotFoundException` com mensagem apropriada.

A opção Listar Utilizadores apresenta todos os utilizadores registados no sistema, exibindo para cada um o ID, nome, email, tipo (Administrador ou Utilizador Comum) e número de casas associadas.

11 Conclusão

O desenvolvimento do sistema DomusControl permitiu aplicar de forma prática os conceitos fundamentais de POO e permitiu-nos utilizar conceitos para a resolução de problemas reais e complexos. A arquitetura desenvolvida revela-se sólida, modular e extensível, facilitando tanto a manutenção do código como a adição de novas funcionalidades sem comprometer a estrutura existente.

A utilização consistente de padrões como Template Method contribuiu significativamente para a qualidade da solução. Garantimos o encapsulamento e a integridade dos dados, enquanto o sistema de exceções personalizadas proporciona tratamento robusto de erros com feedback claro ao utilizador.

Durante o desenvolvimento, foram tomadas decisões conscientes que equilibram funcionalidade e simplicidade. Como trabalho futuro, procuramos melhorar em alguns aspetos como a gestão de permissões de utilizadores, que atualmente se aplicam globalmente ao sistema, sendo mais adequado defini-las por casa, permitindo que um utilizador seja administrador numa casa e utilizador comum noutra. Os escalonamentos são também globais, mas seria ideal cada casa ter as suas configurações independentes, evitando conflitos. Por último, uma interface gráfica tornaria o sistema mais intuitivo.

O desenvolvimento deste projeto permitiu aplicar com sucesso os conceitos fundamentais da Programação Orientada a Objetos, resultando numa solução funcional e extensível. A arquitetura desenvolvida demonstra solidez através da utilização de princípios como encapsulamento, herança e polimorfismo, cumprindo grande parte dos requisitos estabelecidos.